

RECONCILING PROCESS SUPERVISION WITH OUTCOME-BASED CREDIT IN AGENTIC POLICY OPTIMIZATION

Anonymous authors

Paper under double-blind review

ABSTRACT

For language-model agents, outcome-based methods such as GRPO use verified trajectory feedback but assign the same advantage to intermediate decisions, obscuring their different contributions to success or failure. On-policy (self)-distillation supplies denser process supervision by re-evaluating sampled behavior with training-time privileged information (PI). Yet finer supervision does not automatically provide outcome-consistent credit. PI-induced likelihood changes reflect context-dependent policy preferences, but do not specify how outcome-derived advantages should be distributed across decisions. Directly optimizing these signals can introduce objectives that compete with outcome-based reinforcement. Their usefulness also depends on whether the guidance applies to the target interaction and whether token-level changes meaningfully evaluate complete executable actions. We introduce TASPO, a framework for reconciling process supervision and outcome-based optimization through constrained credit redistribution. TASPO aligns guidance from verified successful sibling trajectories with the target interaction and aggregates PI-induced likelihood shifts over executable actions. The resulting relative support reweights the original advantage through positive, bounded, mean-one weights, preserving its sign and trajectory mean without additional environment interaction. Across ALFWorld, Search-QA, and WebShop with three backbone models, TASPO achieves an average gain of 10.6% over GRPO under the same interaction budgets and policy-update counts. Analyses favor aligned guidance and suggest more stable training with action-level allocation.

1 INTRODUCTION

Reinforcement learning trains language-model agents to solve multi-turn tasks through interaction with external environments (Jin et al., 2025; Feng et al., 2025a; Wang et al., 2025). With terminal rewards, GRPO assigns the same group-relative advantage to every decision in a trajectory (Shao et al., 2024), leaving useful actions and avoidable mistakes with identical advantage values. Process supervision can distinguish intermediate behavior, but its preferences may conflict with outcome-derived credit when directly combined (Figure 1(a)). This raises our central question: *how can process supervision refine local credit while keeping reinforcement grounded in verified outcomes?*

Existing approaches refine intermediate credit using environment-state structure or learned process evaluators and value estimates (Lightman et al., 2024; Feng et al., 2025b; Choudhury, 2025). On-policy (self)-distillation (OPSD) offers another route: a teacher branch conditions on training-time privileged information (PI) to supervise responses sampled without PI (Agarwal et al., 2024; Zhang et al., 2026; Zhao et al., 2026; Lu et al., 2026; Yang et al., 2026b). This mechanism turns additional context into dense token-level guidance without a separately trained process reward model. Using it for outcome-based credit assignment, however, requires resolving two connected issues.

First, process preferences alone do not specify outcome credit. PI-induced likelihood shifts indicate how additional context changes policy preferences. A separate distillation loss optimizes these preferences alongside the outcome-based objective. The empirical gradient comparison in Figure 2 shows that these signals need not point in the same direction. Weighting or gating the auxiliary objective regulates its influence (Lu et al., 2026; Zhang et al., 2026; Ding et al., 2026), but

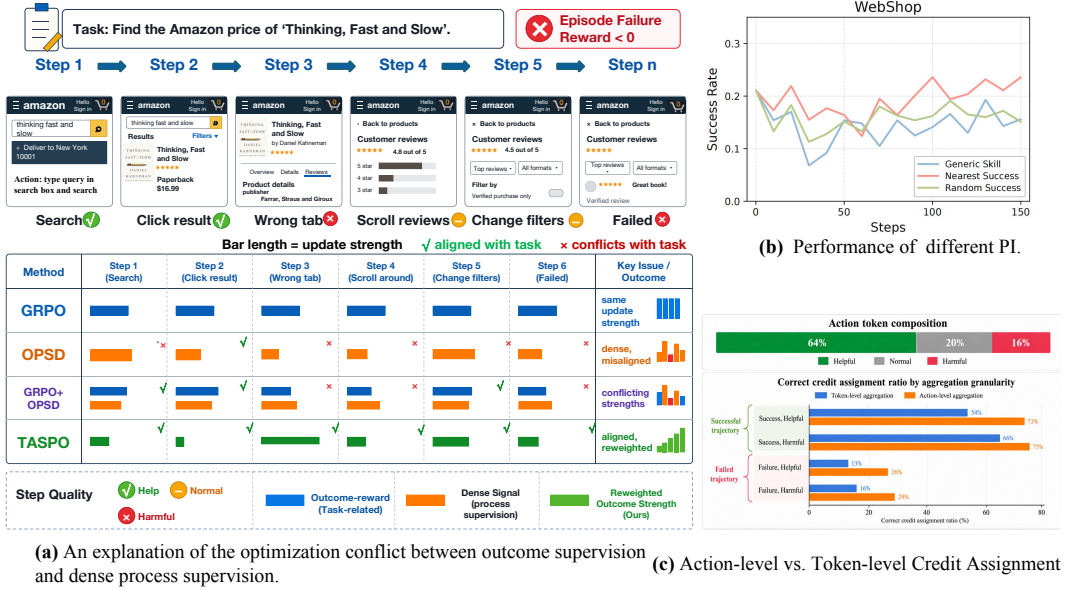


Figure 1: **Reconciling process supervision with outcome-based credit.** (a) Dense process preferences can conflict with outcome-derived credit when directly combined. (b) Self-distillation performance depends on the source and relevance of PI. (c) Action-level aggregation improves credit-assignment accuracy in the measured diagnostic.

leaves open how decisions should share the outcome-derived advantage. We therefore formulate integration as constrained redistribution of the original advantage.

Second, guidance must match the decisions receiving credit. Task-relevant PI can still be inapplicable to the target trajectory’s state or execution path (Xu et al., 2026; Fu et al., 2026). Moreover, token-wise likelihood shifts describe preferences over individual tokens, whereas the environment executes complete actions. Applying these shifts directly can assign inconsistent credit within an action. Figure 1(b,c) shows the sensitivity to PI selection and allocation granularity. Thus, credit redistribution requires guidance applicable to the target interaction and support assessed at the level of executable actions.

We introduce **TASPO**, a framework that reconciles process supervision with outcome-based optimization through **constrained credit redistribution**. The outcome determines the advantage sign; process supervision refines its relative allocation. Using each target’s completed interaction history, a training-time analyzer aligns conditional guidance extracted from verified successful sibling trajectories in the same rollout group. Retained guidance forms a shared PI context; when none applies, TASPO falls back to GRPO. The frozen rollout policy scores sampled responses with and without PI, and aggregates likelihood shifts over executable-action tokens. Positive, bounded, mean-one weights then redistribute the original advantage: stronger relative support receives more positive or less negative credit. Each weight applies to all generated tokens in its turn, preserving the advantage sign and trajectory mean without additional environment interaction.

We evaluate TASPO on ALFWorld (Shridhar et al., 2021), Search-QA (Jin et al., 2025) and WebShop (Yao et al., 2022) with three backbone models. Across nine benchmark–backbone settings, TASPO improves over GRPO by an average of 10.6%. Analyses favor trajectory-aligned guidance over generic skills and direct trajectory reuse. Action-level allocation improves performance and reduces variation across training seeds, while ALFWorld training curves show smoother reward and policy-KL dynamics. Comparisons with standalone OPSD and GRPO+OPSD on ALFWorld further support constrained credit redistribution.

Our contributions are threefold:

1. We formulate the integration of process supervision and outcome feedback as a credit-assignment problem, focusing on optimization consistency and decision alignment.

2. We propose TASPO, which integrates trajectory-aligned, action-level process supervision into outcome-based optimization through constrained credit redistribution.
3. We demonstrate gains over GRPO across three benchmarks and three backbones, supported by analyses of supervision construction, allocation granularity, and optimization dynamics.

2 RELATED WORK

2.1 AGENTIC REINFORCEMENT LEARNING AND CREDIT ASSIGNMENT

Language models are increasingly trained as interactive agents that reason, search, invoke tools, and act over long horizons (Yao et al., 2023; Schick et al., 2023; Liu et al., 2024). Outcome-based reinforcement learning provides a direct way to optimize such agents using environment or verifier feedback (Shao et al., 2024; Guo et al., 2025; Jin et al., 2025; Feng et al., 2025a; Wang et al., 2025; Dong et al., 2025; Feng et al., 2025b). However, sparse trajectory outcomes provide limited information about which intermediate decisions caused success or failure, reflecting the broader delayed-credit problem studied in reinforcement and sequential learning (Andrychowicz et al., 2017; Arjona-Medina et al., 2019; Lightman et al., 2024). Recent agentic methods refine policy optimization using additional structure within sampled trajectories (Dong et al., 2025; Feng et al., 2025b; He et al., 2026). TASPO instead uses training-time privileged information to differentiate decisions within a trajectory while retaining the verified outcome as the source of the overall update direction.

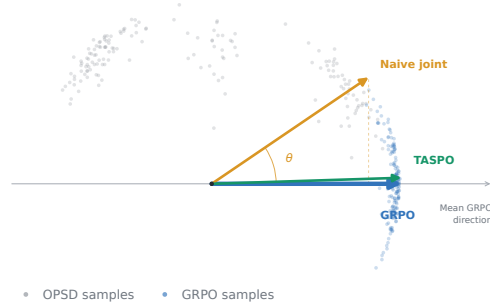


Figure 2: **Gradient alignment with outcome supervision.** TASPO gradients align more closely with the outcome-supervised GRPO reference than those of direct GRPO+OPSP loss combination.

2.2 ON-POLICY SELF-DISTILLATION WITH PRIVILEGED INFORMATION

Knowledge distillation transfers behavioral information from a teacher through token- or sequence-level supervision (Hinton et al., 2015; Kim & Rush, 2016). On-policy distillation further supervises student-generated samples, reducing distribution mismatch between the data being optimized and the current policy (Ross et al., 2011; Agarwal et al., 2024). Recent self-distillation methods construct privileged teachers from additional contexts, reasoning traces, skills, or hindsight available only during training (Zhao et al., 2026; Ye et al., 2026; Yang et al., 2026a; Lu et al., 2026; Wang et al., 2026; Yang et al., 2026b; Wu et al., 2026). Several works further adapt such supervision for finer-grained agent credit, including step-aware, sibling-guided, and counterfactual formulations (Zhang et al., 2026; Ding et al., 2026; Meng & Chen, 2026). A complementary line of work studies when dense on-policy supervision is reliable, considering teacher–student compatibility, token importance, trajectory position, curriculum, and trust-region constraints (Fu et al., 2026; Li et al., 2026b; Xu et al., 2026; Xing et al., 2026; Li et al., 2026a; Xie et al., 2026). TASPO focuses on two distinct questions: whether trajectory-derived guidance is applicable to the target decision, and how such guidance should refine credit without overriding verified outcomes. It therefore aligns source-grounded PI to each target pre-action context and uses the resulting signal only to redistribute outcome-derived credit across executable actions.

3 METHOD

TASPO uses process supervision to distinguish actions that otherwise share the same outcome advantage. To make this distinction useful for policy optimization, we first specify how process information may modify the advantage, then construct guidance applicable to the target interaction and compare its support across executable actions. The resulting scores determine the advantage redistribution shown in Figure 3.

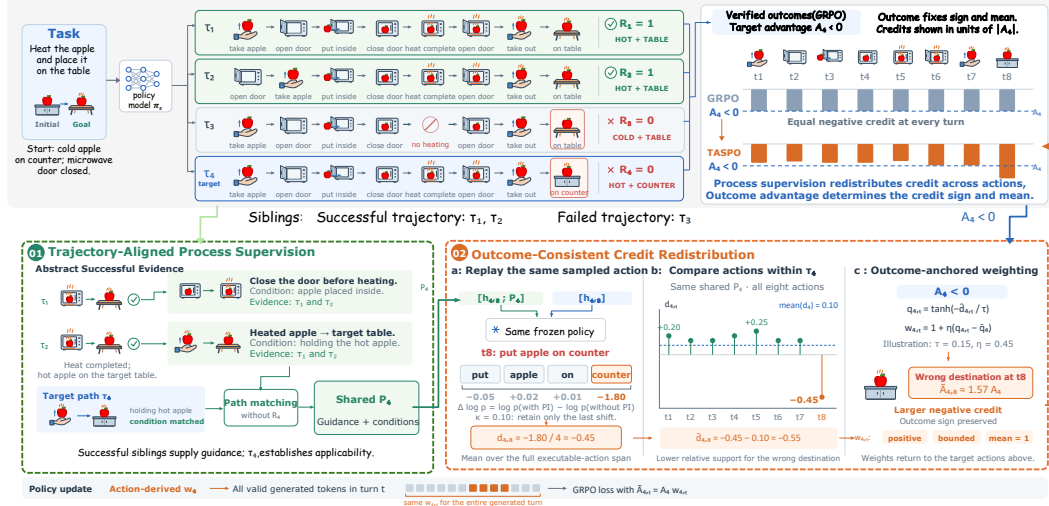


Figure 3: **Overview of TASPO.** TASPO reconciles process supervision with outcome-based credit using shared, trajectory-aligned PI from verified successful siblings. Action-level likelihood shifts yield positive, bounded, mean-one advantage weights, refining local credit while preserving the original advantage’s sign and trajectory mean.

3.1 OUTCOME-CONSTRAINED CREDIT ASSIGNMENT

For a task x , the rollout policy $\pi_{\theta_{\text{old}}}$ samples K trajectories $\mathcal{G}_x = \{\tau_i\}_{i=1}^K$. At turn t , it generates response y_{it} from history h_{it} , and the environment executes action a_{it} . The verified terminal reward R_i yields the GRPO advantage $A_i = (R_i - \mu_{\mathcal{G}_x}) / (\sigma_{\mathcal{G}_x} + \delta)$, where $\mu_{\mathcal{G}_x}$ and $\sigma_{\mathcal{G}_x}$ are the group reward mean and standard deviation, and $\delta > 0$ stabilizes normalization. GRPO uses the same A_i across all T_i turns. We let process supervision differentiate these turns by reweighting A_i within the trajectory:

$$\tilde{A}_{it} = A_i w_{it}, \quad 1 - \epsilon_w \leq w_{it} \leq 1 + \epsilon_w, \quad \frac{1}{T_i} \sum_{t=1}^{T_i} w_{it} = 1, \quad (1)$$

with $0 \leq \epsilon_w < 1$. We retain positive weights so that each turn inherits the original advantage sign, and require mean-one weights to preserve the average advantage. Guidance can still be applied to unsuitable states, and likelihood increases need not indicate task progress. We therefore bound each turn’s adjustment by $\epsilon_w |A_i|$. Process supervision thus enters the GRPO advantage without a separate PI-matching objective. These guarantees concern scalar advantages; the parameter-gradient direction can still change.

3.2 TRAJECTORY-ALIGNED PROCESS SUPERVISION

We first need guidance that applies to the target’s actual interaction. A training-time analyzer extracts conditional guidance $\mathcal{E}_x$ from verified successful trajectories $\mathcal{S}_x \subseteq \mathcal{G}_x$, such as task requirements, action preconditions, and ordering constraints. Each item cites supporting source actions and observations. These conditions may differ across rollouts, so source evidence alone is insufficient to establish applicability.

For target τ_i , we exclude items supported only by that target to obtain $\mathcal{E}_{x,-i}$. The analyzer then checks the remaining guidance against the task and the target’s complete recorded action–observation history $\mathcal{H}_i$. Source interactions support the guidance itself; target interactions provide the basis for assessing its relevance. Retained or reformulated statements cite both. Alignment occurs after rollout without receiving the target’s explicit terminal reward or success label. The resulting statements form a compact privileged context:

$$P_i = \text{Compose}(\text{Align}(\mathcal{E}_{x,-i}, x, \mathcal{H}_i)). \quad (2)$$

We reuse this P_i across turns to keep the guidance fixed during action comparison, while retaining each turn’s original history and response prefix. If no statements pass the evidence checks, P_i is empty. Appendix A.2 details the prompts, evidence checks, and context budget.

3.3 RELATIVE ACTION SCORING

With P_i fixed, we use the privileged rescoring mechanism of OPSD to measure how the guidance changes the likelihood of sampled behavior. We focus on the tokens specifying the action submitted to the environment, so that the scoring unit matches the decision whose credit we adjust. Let $\mathcal{M}_{it}$ denote these token positions in y_{it} . The frozen rollout policy computes

$$\Delta_{itk} = \text{sg}[\log \pi_{\theta_{\text{old}}}(y_{itk} \mid P_i, h_{it}, y_{it, < k}) - \log \pi_{\theta_{\text{old}}}(y_{itk} \mid h_{it}, y_{it, < k})], \quad (3)$$

where sg stops gradients. A positive Δ_{itk} means that adding PI makes the sampled token more likely. Let $\mathcal{I}_i$ contain turns with available scoring inputs, a complete nonempty action span, and finite log probabilities and shifts for every action token.

A practical issue is that a single extreme shift can dominate the action mean and distort comparisons with other actions. We therefore clip token shifts before averaging and center the resulting action scores:

$$\begin{aligned} d_{it} &= \frac{1}{|\mathcal{M}_{it}|} \sum_{k \in \mathcal{M}_{it}} \text{clip}(\Delta_{itk}, -c_{\Delta}, c_{\Delta}), \\ \hat{d}_{it} &= d_{it} - \frac{1}{|\mathcal{I}_i|} \sum_{u \in \mathcal{I}_i} d_{iu}, \quad t \in \mathcal{I}_i. \end{aligned} \quad (4)$$

Here, $c_{\Delta} > 0$ limits individual token shifts, and averaging avoids directly scaling the score with action length. Centering provides a reference within the trajectory: $\hat{d}_{it} > 0$ means the action’s score exceeds the mean over scorable turns. We use this relative support to allocate the outcome advantage.

3.4 ADVANTAGE REDISTRIBUTION AND POLICY OPTIMIZATION

We note that on a negative-advantage trajectory, larger weights produce more negative credit. Simply increasing weights with PI support would therefore penalize better-supported actions more heavily. To obtain the intended allocation for both advantage signs, we use

$$q_{it} = \tanh\left(\frac{\text{sign}(A_i)\hat{d}_{it}}{\tau}\right), \quad t \in \mathcal{I}_i, \quad (5)$$

for $A_i \neq 0$ and $|\mathcal{I}_i| \geq 2$, where $\tau > 0$ controls sensitivity. We then convert these scores into deviations from unit weights:

$$w_{it} = 1 + \eta(q_{it} - \bar{q}_i), \quad \bar{q}_i = \frac{1}{|\mathcal{I}_i|} \sum_{u \in \mathcal{I}_i} q_{iu}, \quad \eta = \frac{\epsilon_w}{2}. \quad (6)$$

The second centering is necessary because tanh need not preserve the zero mean of $\hat{d}_{it}$. It makes the weights average to one over $\mathcal{I}_i$, while $q_{it} \in [-1, 1]$ gives $|w_{it} - 1| \leq \epsilon_w$. This favors better-supported actions with relatively more positive credit when $A_i > 0$ and less negative credit when $A_i < 0$. Appendix B.1 derives the weighting rule and establishes its credit-preservation properties.

Unscorable turns retain $w_{it} = 1$, allowing the remaining turns to participate. If shared PI cannot be constructed, $A_i = 0$, or fewer than two turns are scorable, all weights remain one; fewer than two scores provide no relative comparison. Since excluded turns retain unit weights, redistribution within $\mathcal{I}_i$ also preserves the mean over the complete trajectory, satisfying Eq. 1.

Finally, we apply each action-derived advantage to the entire response that produced the action. Let $\mathcal{R}_{it}$ contain policy-generated token positions, including reasoning and action tokens but excluding environment observations. The policy objective is

$$\mathcal{L}_{\text{policy}} = \frac{1}{B} \sum_{i=1}^B \frac{1}{T_i} \sum_{t=1}^{T_i} \frac{1}{|\mathcal{R}_{it}|} \sum_{k \in \mathcal{R}_{it}} \ell_{itk}^{\text{GRPO}}(\theta; \tilde{A}_{it}), \quad (7)$$

Table 1: Main results on ALFWorld, Search-QA, and WebShop across three backbone models. ALFWorld reports task-family success rates, Search-QA reports dataset accuracy and WebShop reports Score and success rate (Succ.), all in percent. Avg. is the unweighted mean of the six ALFWorld task-family scores or the seven Search-QA dataset scores. **Bold** and underlined values mark the best and second-best distinct values within each backbone.

Method	ALFWorld							Search-QA							WebShop		
	Pick	Look	Clean	Heat	Cool	Pick2	Avg.	NQ	Triv	Pop	Hotp	2Wk	MuS	Bam	Avg.	Score	Succ.
Qwen2.5-3B-Instruct																	
Vanilla	44.4	11.1	6.2	15.4	28.6	12.5	19.7	24.6	48.1	31.0	26.3	25.3	7.2	59.7	31.7	6.7	0.8
OPSD	48.8	41.7	16.7	0.0	15.8	16.7	23.3	0.1	0.1	0.1	0.0	0.0	0.0	0.0	0.0	11.3	3.1
GRPO	87.9	68.8	81.8	58.3	65.4	68.4	71.8	39.3	60.6	41.1	37.4	34.6	15.4	26.4	36.4	79.8	63.3
GiGPO	94.3	72.7	93.8	<u>90.1</u>	64.3	<u>89.5</u>	84.1	43.0	60.5	46.9	36.2	35.5	12.1	64.1	42.6	83.8	70.3
SDAR	<u>97.1</u>	62.5	100.0	61.9	75.0	84.2	80.1	44.8	58.1	44.3	38.6	36.2	15.7	<u>66.1</u>	43.4	<u>85.0</u>	68.0
StepOPSD	<u>97.1</u>	66.7	87.0	79.1	78.9	95.0	84.0	43.6	61.2	43.8	39.2	<u>38.1</u>	15.8	64.5	43.7	—	—
OPID	92.7	100.0	88.9	70.0	84.2	70.0	<u>84.3</u>	<u>45.9</u>	<u>61.4</u>	<u>45.7</u>	<u>40.7</u>	38.8	<u>16.4</u>	<u>66.1</u>	<u>45.0</u>	<u>85.0</u>	<u>74.2</u>
TASPO	97.9	<u>77.5</u>	<u>95.8</u>	95.8	<u>79.6</u>	70.8	86.2	46.8	65.5	44.9	41.1	37.4	19.1	70.2	46.4	88.5	78.1
Qwen2.5-7B-Instruct																	
Vanilla	36.1	22.2	3.1	0.0	0.0	0.0	10.2	25.2	50.8	29.5	29.0	29.0	10.4	63.7	33.9	5.9	1.6
GRPO	91.2	70.0	90.9	78.6	64.7	66.7	77.0	46.3	63.8	47.2	44.6	<u>44.3</u>	19.3	71.4	48.1	85.7	75.0
GiGPO	97.4	<u>82.3</u>	95.8	83.7	86.9	78.7	87.5	46.3	64.1	46.5	41.6	43.6	18.9	68.9	47.1	84.4	72.8
OPSD	50.0	60.0	22.7	21.4	17.6	9.5	30.2	8.8	8.6	17.5	2.5	4.2	0.5	1.2	6.2	4.5	2.3
SDAR	94.7	75.0	100.0	86.7	68.2	78.9	83.9	46.3	63.5	<u>48.2</u>	43.8	48.4	<u>19.6</u>	<u>73.0</u>	49.0	89.4	82.8
OPID	100.0	81.8	<u>97.1</u>	100.0	80.8	<u>80.0</u>	<u>90.0</u>	<u>48.8</u>	<u>65.6</u>	46.8	46.1	42.7	21.7	72.6	49.2	85.3	<u>79.7</u>
TASPO	<u>98.2</u>	84.8	94.4	<u>98.3</u>	<u>83.3</u>	80.7	90.1	49.6	65.9	49.3	<u>45.1</u>	43.3	18.4	76.7	49.8	<u>87.7</u>	76.4
Qwen3-1.7B-Instruct																	
Vanilla	25.0	22.2	3.1	0.0	21.4	4.2	12.7	29.4	46.9	37.0	23.5	19.6	6.4	10.5	24.8	46.5	4.7
OPSD	26.3	33.3	9.1	0.0	4.5	5.3	13.1	4.2	8.3	4.6	6.6	15.3	0.7	1.2	5.8	47.4	9.3
GRPO	59.3	46.7	12.5	68.4	16.7	36.8	40.1	<u>43.0</u>	<u>59.7</u>	47.4	<u>37.9</u>	<u>37.4</u>	<u>13.6</u>	<u>66.1</u>	<u>43.6</u>	55.8	43.0
GiGPO	79.9	60.3	62.6	<u>62.5</u>	59.1	65.1	<u>64.9</u>	41.1	59.3	45.1	34.1	32.7	10.5	64.5	41.0	70.0	45.3
SDAR	73.5	25.0	<u>76.9</u>	33.3	40.0	36.8	47.6	39.7	58.9	45.3	35.9	35.5	12.6	65.3	41.9	76.8	58.6
StepOPSD	64.7	44.4	56.5	60.9	42.1	55.0	53.9	40.5	58.6	<u>45.6</u>	34.9	29.8	10.8	64.5	40.7	—	—
OPID	65.9	72.7	66.7	40.0	<u>63.2</u>	45.0	58.9	38.1	58.1	43.4	35.5	31.7	11.7	64.5	40.4	79.6	<u>64.8</u>
TASPO	<u>74.2</u>	<u>70.8</u>	78.4	60.3	73.3	<u>60.7</u>	69.6	46.1	62.6	45.4	40.5	43.7	19.7	68.5	46.6	<u>77.6</u>	67.1

where B is the number of trajectories and ℓ^{GRPO} is the GRPO token policy loss with the supplied advantage. Averaging within turns and then across turns gives each turn equal base weight before redistribution. Any KL regularization follows the GRPO baseline. PI construction and rescore use completed rollouts during training and require no additional environment interaction.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Benchmarks and evaluation. We evaluate TASPO on three long-horizon agent domains: ALFWorld, WebShop, and Search-QA, following evaluation protocols used in recent agentic RL and distillation studies (Lu et al., 2026; Zhang et al., 2026; Yang et al., 2026b; Wu et al., 2026). For ALFWorld, we report success rates on the six task families—Pick, Look, Clean, Heat, Cool, and Pick2—together with their macro-average. Main results use the 140-task seen split, and matched reruns are additionally evaluated on the 134-task unseen split. For WebShop, we evaluate on 128 fixed tasks and report normalized task score and success rate. Search-QA comprises NQ, TriviaQA, PopQA, HotpotQA, 2WikiMultiHopQA, MuSiQue, and Bamboogle. We train on NQ and HotpotQA and report exact-match accuracy on all seven datasets and their macro-average.

Baselines. We compare against the instruction-tuned backbone without post-training (Vanilla), GRPO, and recent methods using training-time skills or privileged supervision, including SDAR, StepOPSD, OPID (Lu et al., 2026; Zhang et al., 2026; Yang et al., 2026b). For controlled comparisons, we rerun methods with the same backbone, environment-interaction budget, number of policy updates, and evaluation tasks. Results taken directly from prior work are reported separately for reference, while our main comparisons are based on matched reruns.

Training configuration. We evaluate Qwen2.5-3B-Instruct, Qwen2.5-7B-Instruct, and Qwen3-1.7B-Instruct. Unless otherwise stated, analyses use Qwen2.5-3B-Instruct. For TASPO and baseline reruns, we use eight rollouts per task and 150 policy updates. Task batch sizes are 16 for ALFWorld and WebShop and 128 for Search-QA, with interaction horizons of 50, 15, and 4, respectively. We use a learning rate of 10^{-6} and average these runs over three training seeds. TASPO uses a frozen DeepSeek-V4-Pro analyzer to construct and align privileged guidance, with default $\tau = 0.5$ and $\epsilon_w = 0.4$. Evaluation uses fixed task and decoding seeds. Appendix C details settings and result sources; Appendix D.4 reports computational cost.

4.2 MAIN RESULTS

Consistent gains across backbones. Table 1 shows that TASPO consistently improves over GRPO across all three benchmarks and backbones. On ALFWorld, macro-averaged success increases by 14.4, 13.0, and 29.5 percentage points on Qwen2.5-3B, Qwen2.5-7B, and Qwen3-1.7B, respectively. TASPO also achieves the highest Search-QA macro-average and the best or tied-best ALFWorld macro-average across all three backbones, while improving both WebShop metrics over GRPO. These results support the effectiveness of integrating process supervision through outcome-based credit allocation across different tasks and model scales.



Figure 4: Training progress on ALFWorld. TASPO improves faster and reaches a higher final success rate than GRPO and strong baseline SDAR.

Faster policy improvement. Figure 4 shows that TASPO improves faster after the initial exploration stage and reaches a higher final success rate than GRPO and SDAR under the same policy-update budget. We attribute this to two reasons: First, Figure 2 illustrates how an independent distillation objective can diverge from outcome-based optimization. TASPO incorporates process supervision through constrained credit redistribution, refining action-level learning signals while preserving outcome-derived advantage signs. Second, PI is constructed from verified successful siblings within the current rollout group. Early in training, limited successful experience restricts the availability of applicable guidance, and TASPO falls back to GRPO when none is available. As the policy improves, more successful rollouts provide additional evidence from which applicable guidance can be constructed. This can establish a positive feedback loop: policy improvement expands the evidence available for process supervision, which in turn supports more effective credit allocation and further policy improvement. In appendix E we illustrate error recovery and task-constraint handling in ALFWorld and WebShop.

4.3 ANALYSIS OF PRIVILEGED INFORMATION

PI construction matters. Table 2 compares PI construction strategies under the same training protocol. Successful trajectory reuse provides larger gains than generic skills, suggesting the value of concrete interaction evidence. TASPO’s abstracted, trajectory-aligned PI further achieves 86.3% success, exceeding Nearest Success by 5.2%. Its construction combines conditional guidance extraction with target-trajectory matching, addressing source-specific decisions that may be inapplicable to the target’s interaction path. These results support transforming successful experience into decision-applicable supervision before using it to refine outcome-based credit. Appendix D.1 further examines target alignment while keeping guidance extraction fixed.

Analyzer robustness. We also vary the training-time analyzer while keeping the PI construction procedure and training settings fixed (Table 3). Across the three tested analyzers, PI coverage ranges from 71.6% to 72.4%, and success rates from 85.7% to 86.3%. These small differences suggest that TASPO’s performance has limited sensitivity to analyzer choice in this setting.

4.4 TOKEN-LEVEL VERSUS ACTION-LEVEL ALLOCATION

Action-level allocation improves performance. We compare the two allocation units using identical PI construction and trajectory alignment, with the same optimization budget and training configuration.

Table 2: **PI construction on ALFWorld.** Macro-averaged success rates with Qwen2.5-3B-Instruct. Alignment checks whether source guidance applies to the target trajectory’s recorded interaction context.

PI construction	Source	Alignment	ALF Avg. (%)
GRPO	–	–	71.8
Generic Skill	Task skill	×	76.9
Random Success	Successful trajectory	×	79.2
Nearest Success	Similar successful trajectory	×	81.1
TASPO	Guidance from successful siblings	✓	86.2

Table 3: **Analyzer robustness on ALFWorld.** PI coverage and macro-averaged success rates with different training-time analyzers, using Qwen2.5-3B-Instruct.

Analyzer	PI coverage (%)	ALF Avg. (%)
DeepSeek-V4-Pro	72.4	86.2
GLM5.2	71.6	85.7
Qwen3.5-35B-A3B	72.1	86.0

Across three training seeds, action-level allocation improves performance on all three benchmarks (Table 4). Gains are 6.6% on ALFWorld and 6.0% on WebShop, with a smaller 0.5% gain on Search-QA. On ALFWorld, the reported cross-seed variation decreases from 2.9% to 1.4% , and successful trajectories require fewer environment steps on average: 9.2 compared with 11.4. Appendix D.3 separately compares applying action-derived weights to action tokens or the full response.

Action-level allocation also yields steadier training dynamics. Figure 5 shows smoother reward progression and fewer pronounced KL spikes than token-level allocation. Using a shared action-level credit coefficient prevents individual tokens within the same executable action from receiving different PI-derived credit coefficients. The observed dynamics are consistent with the benefit of reducing these within-action differences in credit allocation.

4.5 CREDIT REWEIGHTING VERSUS DISTILLATION

Reweighting outperforms distillation. Table 5 compares different ways of incorporating additional supervision. Standalone OPSD underperforms GRPO substantially (28.1% vs. 71.8%), whereas GRPO + OPSD improves success to 81.7%. TASPO reaches 86.3%, exceeding the joint objective and the best tested OPD configuration by 4.6% and 2.7%, respectively. These results are consistent with the supervision–credit discrepancy illustrated in Figure 2: an independent distillation objective can favor updates that differ from outcome-based reinforcement. TASPO converts PI into bounded action weights on the original trajectory advantage, preserving its sign while refining credit across decisions. Appendix D.2 further compares distillation and credit redistribution under the same aligned-PI construction protocol.

Table 5: **Supervision strategies on ALFWorld.** Macro-averaged success rates with Qwen2.5-3B-Instruct. Self + PI denotes the PI-conditioned roll-out policy used for scoring.

Method	Teacher	ALF Avg. (%)
GRPO	–	71.8
OPSD	Self	23.3
OPD	Qwen2.5-7B	76.4
OPD	Qwen3-32B	83.6
GRPO + OPSD	Self	81.7
TASPO	Self + PI	86.2

4.6 HYPERPARAMETER ANALYSIS

In this section we conduct hyperparameter sensitivity on ALFWorld with Qwen2.5-3B-Instruct, varying the redistribution amplitude and comparing action-level with token-level allocation.

Redistribution strength. For action-level allocation, we vary ϵ_w over $\{0.4, 0.6, 0.8\}$. As shown in Table 6, the default $\epsilon_w = 0.4$ achieves the highest success rate among the tested values, while both larger settings perform worse. All three settings preserve the advantage sign and trajectory mean, but

Table 4: **Effect of credit allocation granularity.** Results with Qwen2.5-3B-Instruct. ALF Avg. and Search Avg. are macro averages; performance scores are in percent. ALF Steps is the mean number of environment steps among successful ALFWorld trajectories.

Allocation unit	ALF Avg. $\uparrow$	Search Avg. $\uparrow$	WebShop Succ. $\uparrow$	ALF Steps $\downarrow$
Token-level	79.7 ± 2.9	42.9 ± 1.7	72.1 ± 3.9	11.4
Action-level	86.3 ± 1.4	46.4 ± 1.2	78.1 ± 2.6	9.2

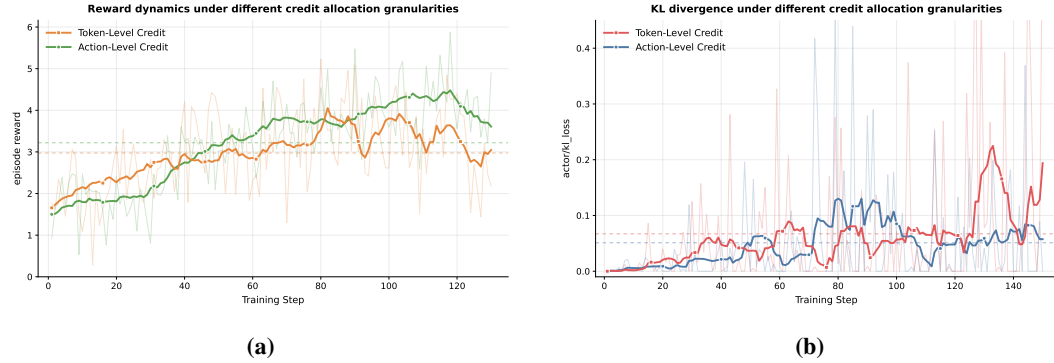


Figure 5: **Optimization dynamics under different credit allocation granularities.** (a) Action-level allocation achieves smoother reward improvement and a higher final reward compared with token-level allocation. (b) Token-level allocation introduces larger policy-update fluctuations, while action-level allocation maintains more stable KL dynamics.

these constraints do not ensure that PI-derived support accurately reflects each action’s contribution to task completion. A larger ϵ_w allows greater deviations from the original advantage and can therefore amplify errors in relative credit allocation. The observed pattern favors a moderate redistribution amplitude in this setting.

Allocation granularity. As shown in Table 6, action-level allocation achieves 86.2% success, exceeding the token-level variant’s 79.7% by 6.5 percentage points. Figure 5 further shows smoother reward improvement and fewer pronounced KL spikes. These observations support organizing credit around executable actions: TASPO aggregates PI-induced likelihood shifts over action tokens and applies a shared coefficient to the corresponding response, keeping the credit weights consistent within each action.

Table 6: **Effect of credit allocation settings.** Macro-averaged success on ALFWorld after 150 updates with Qwen2.5-3B-Instruct. We fix $\tau = 0.5$ and $c_\Delta = 2.0$.

Allocation unit	ϵ_w	ALF Avg. (%)
Action-level	0.2	83.1
Action-level	0.4	86.2
Action-level	0.6	84.2
Action-level	0.8	83.5
Token-level	0.4	79.7

5 CONCLUSION

We introduced TASPO to reconcile process supervision with outcome-based credit in agentic policy optimization. Our central idea is to use process information to determine how outcome-derived credit is distributed across decisions. TASPO aligns conditional guidance from verified successful siblings with the target interaction and aggregates PI-induced likelihood shifts over executable actions. Positive, bounded, mean-one weights then refine the original advantage while preserving its sign and trajectory mean, without additional environment interaction. Across three benchmarks and three backbones, TASPO consistently improves over GRPO, with ALFWorld learning curves also showing faster policy improvement. Further analyses support the roles of trajectory alignment, action-level allocation, and credit redistribution. Together, these findings suggest that effective process supervision requires both applicable guidance and a way to incorporate it into learning that remains grounded in verified outcomes.

REFERENCES

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2306.13649>.
- Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hind-sight experience replay. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/453fadbd8a1a3af50a9df4df899537b5-Abstract.html>.
- Jose A. Arjona-Medina, Michael Gillhofer, Michael Widrich, Thomas Unterthiner, Johannes Brandstetter, and Sepp Hochreiter. RUDDER: Return decomposition for delayed rewards. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://papers.nips.cc/paper_files/paper/2019/hash/16105fb9cc614fc29e1bda00dab60d41-Abstract.html.
- Sanjiban Choudhury. Process reward models for llm agents: Practical framework and directions, 2025. URL <https://arxiv.org/abs/2502.10325>.
- Tianyu Ding, Jianhong Xin, and Juan Pablo De la Cruz Weinstein. Keep policy gradient in charge: Sibling-guided credit distillation for long-horizon tool-use agents. *arXiv preprint arXiv:2606.12634*, 2026. URL <https://arxiv.org/abs/2606.12634>.
- Guanting Dong, Hangyu Mao, Kai Ma, Licheng Bao, Yifei Chen, Zhongyuan Wang, Zhongxia Chen, Jiazhen Du, Huiyang Wang, Fuzheng Zhang, Guorui Zhou, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Agentic reinforced policy optimization. *arXiv preprint arXiv:2507.19849*, 2025. URL <https://arxiv.org/abs/2507.19849>.
- Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjuan Zhong. ReTool: Reinforcement learning for strategic tool use in LLMs. *arXiv preprint arXiv:2504.11536*, 2025a. URL <https://arxiv.org/abs/2504.11536>.
- Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for LLM agent training. In *Advances in Neural Information Processing Systems*, volume 38, 2025b. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/420c9f777c0b4f78d515e53cf74d58b2-Abstract-Conference.html.
- Yuqian Fu, Haohuan Huang, Kaiwen Jiang, Jiakai Liu, Zhuo Jiang, Yuanheng Zhu, and Dongbin Zhao. Revisiting on-policy distillation: Empirical failure modes and simple fixes. *arXiv preprint arXiv:2603.25562*, 2026. URL <https://arxiv.org/abs/2603.25562>.
- Daya Guo et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:633–638, 2025. doi: 10.1038/s41586-025-09422-z. URL <https://www.nature.com/articles/s41586-025-09422-z>.
- Langzhou He, Junyou Zhu, Yue Zhou, Zhengyao Gu, Junhua Liu, Wei-Chieh Huang, Henry Peng Zou, David Wipf, Philip S. Yu, and Qitian Wu. Resolving action bottleneck: Agentic reinforcement learning informed by token-level energy. *arXiv preprint arXiv:2605.14558*, 2026. URL <https://arxiv.org/abs/2605.14558>.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. URL <https://arxiv.org/abs/1503.02531>.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025. URL <https://arxiv.org/abs/2503.09516>.

- Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 1317–1327. Association for Computational Linguistics, 2016. doi: 10.18653/v1/D16-1139. URL <https://aclanthology.org/D16-1139/>.
- Gengsheng Li, Mao Zheng, Mingyang Song, Ruiqi Liu, Tianyu Yang, Jie Sun, Qiyong Zhong, Haiyun Guo, Junfeng Fang, Dan Zhang, and Jinqiao Wang. On-policy distillation with curriculum turn-level guidance for multi-turn agents. *arXiv preprint arXiv:2606.15912*, 2026a. URL <https://arxiv.org/abs/2606.15912>.
- Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huan-gao Gao, Wenkai Yang, Zhiyuan Liu, and Ning Ding. Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe. *arXiv preprint arXiv:2604.13016*, 2026b. URL <https://arxiv.org/abs/2604.13016>.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s Verify Step by Step. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2305.20050>.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2308.03688>.
- Zhengxi Lu, Zhiyuan Yao, Zhuowen Han, Zi-Han Wang, Jinyang Wu, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Self-distilled agentic reinforcement learning. *arXiv preprint arXiv:2605.15155*, 2026. URL <https://arxiv.org/abs/2605.15155>.
- Zibin Meng and Kani Chen. CRAFT: Counterfactual credit assignment from free sibling rollouts for self-distilled agentic reinforcement learning. *arXiv preprint arXiv:2606.29476*, 2026. URL <https://arxiv.org/abs/2606.29476>.
- Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, volume 15 of *Proceedings of Machine Learning Research*, pp. 627–635, 2011. URL <https://proceedings.mlr.press/v15/ross11a.html>.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2302.04761>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021. URL <https://arxiv.org/abs/2010.03768>.
- Hao Wang, Guozhi Wang, Han Xiao, Yufeng Zhou, Yue Pan, Jichao Wang, Ke Xu, Yafei Wen, Xiaohu Ruan, Xiaoxin Chen, and Honggang Qi. Skill-SD: Skill-conditioned self-distillation for multi-turn LLM agents. *arXiv preprint arXiv:2604.10674*, 2026. URL <https://arxiv.org/abs/2604.10674>.
- Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Yiping Lu, Kyunghyun Cho, Jiajun Wu,

- Fei-Fei Li, Lijuan Wang, Yejin Choi, and Manling Li. RAGEN: Understanding self-evolution in LLM agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025. URL <https://arxiv.org/abs/2504.20073>.
- Jinyang Wu, Shuo Yang, Zhengxi Lu, Fan Zhang, Yuhao Shen, Lang Feng, Haoran Luo, Zheng Lian, Shuai Zhang, Zhengqi Wen, and Jianhua Tao. SEED: Self-evolving on-policy distillation for agentic reinforcement learning. *arXiv preprint arXiv:2607.14777*, 2026. URL <https://arxiv.org/abs/2607.14777>.
- Yan Xie, Sijie Zhu, Tiansheng Wen, Bo Chen, and Yifei Wang. On the position bias of on-policy distillation. *arXiv preprint arXiv:2606.22600*, 2026. URL <https://arxiv.org/abs/2606.22600>.
- Xingrun Xing, Haoqing Wang, Boyan Gao, Ziheng Li, and Yehui Tang. Trust region on-policy distillation. *arXiv preprint arXiv:2606.01249*, 2026. URL <https://arxiv.org/abs/2606.01249>.
- Yuanda Xu, Hejian Sang, Zhengze Zhou, Ran He, Zhipeng Wang, and Alborz Geramifard. TIP: Token importance in on-policy distillation. *arXiv preprint arXiv:2604.14084*, 2026. URL <https://arxiv.org/abs/2604.14084>.
- Chenxu Yang, Chuanyu Qin, Qingyi Si, Minghui Chen, Naibin Gu, Dingyu Yao, Zheng Lin, Weiping Wang, Jiaqi Wang, and Nan Duan. Self-distilled RLVR. *arXiv preprint arXiv:2604.03128*, 2026a. URL <https://arxiv.org/abs/2604.03128>.
- Shuo Yang, Jinyang Wu, Zhengxi Lu, Yuhao Shen, Fan Zhang, Lang Feng, Shuai Zhang, Haoran Luo, Zheng Lian, Zhengqi Wen, and Jianhua Tao. OPID: On-policy skill distillation for agentic reinforcement learning. *arXiv preprint arXiv:2606.26790*, 2026b. URL <https://arxiv.org/abs/2606.26790>.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems*, volume 35, pp. 20744–20757, 2022. URL <https://arxiv.org/abs/2207.01206>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. On-policy context distillation for language models. *arXiv preprint arXiv:2602.12275*, 2026. URL <https://arxiv.org/abs/2602.12275>.
- Yanfei Zhang, Xu Lin, and Chenglin Wu. StepOPSD: Step-aware online preference distillation for agent reinforcement learning. *arXiv preprint arXiv:2605.27140*, 2026. URL <https://arxiv.org/abs/2605.27140>.
- Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026. URL <https://arxiv.org/abs/2601.18734>.

A METHOD AND IMPLEMENTATION DETAILS

A.1 TRAINING ALGORITHM

Algorithm 1 specifies one training iteration. The analyzer and rollout snapshot are frozen; only the policy is updated. PI conditions rescoring, while policy optimization and evaluation use the original contexts.

Algorithm 1 One TASPO training iteration

Require: Policy π_θ , frozen analyzer, task batch $\mathcal{B}$, group size K , clipping threshold $c_\Delta > 0$, temperature $\tau > 0$, range $0 \leq \epsilon_w < 1$

- 1: Freeze the rollout snapshot $\theta_{\text{old}} \leftarrow \theta$
- 2: **for** each task $x \in \mathcal{B}$ **do**
- 3: Sample K trajectories with $\pi_{\theta_{\text{old}}}$; compute rewards R_i and group-relative advantages A_i
- 4: Extract $\mathcal{E}_x$ from verified successful trajectories; use $\mathcal{E}_x = \emptyset$ if none are available
- 5: **for** each target trajectory τ_i **do**
- 6: Initialize $w_{it} \leftarrow 1$ for every turn
- 7: Exclude evidence supported only by τ_i
- 8: Align evidence with the complete target path, excluding explicit outcome labels; validate and compose shared P_i
- 9: **if** $P_i \neq \emptyset$ and $A_i \neq 0$ **then**
- 10: Construct paired inputs, parse actions, and rescore available turns using $\pi_{\theta_{\text{old}}}$
- 11: Collect scorable turns $\mathcal{I}_i$ using the checks in Appendix A.3
- 12: **if** $|\mathcal{I}_i| \geq 2$ **then**
- 13: Compute clipped action means d_{it} using Eq. equation A.1
- 14: $\hat{d}_{it} \leftarrow d_{it} - \text{mean}_{u \in \mathcal{I}_i} d_{iu}$
- 15: $q_{it} \leftarrow \tanh(\text{sign}(A_i) \hat{d}_{it} / \tau)$
- 16: $w_{it} \leftarrow 1 + \frac{\epsilon_w}{2} (q_{it} - \text{mean}_{u \in \mathcal{I}_i} q_{iu})$
- 17: **end if**
- 18: **end if**
- 19: Assign detached $\tilde{A}_{it} = A_i w_{it}$ to all policy-generated tokens in each turn
- 20: **end for**
- 21: **end for**
- 22: Update θ on the original policy contexts using Eq. equation 7

Without a verified success in the group, TASPO retains the original advantages. It provides no additional process guidance for initial exploration in this regime.

A.2 ANALYZER PROTOCOL AND PROMPTS

Source evidence and target applicability. Extraction identifies source-supported guidance; alignment checks its applicability to the target path. Candidates must retain support from another successful trajectory. Aligned statements cite both source and target evidence and form one shared P_i .

Inputs and outcome visibility. Both stages receive compact traces as JSON user messages. Extraction uses verified successes; alignment receives the target task, all recorded steps, and allowed external evidence. The target payload excludes `verified.success`, `episode.reward`, and precomputed advantages. Actual observations remain unchanged and may reveal task completion; the exclusion applies to explicit metadata.

Prompt 1: Source evidence extraction

You are a conservative training-time evidence analyst for a language-model agent. You receive only environment-verified successful trajectories for one task. Abstract reusable guidance without inventing states, observations, actions, or outcomes.

Rules:

1. Extract task requirements, state-conditioned progress rules, constraints, and valid alternative paths. Do not turn one trajectory into a single mandatory plan.
-

Prompt 1: Source evidence extraction (continued)

2. Every item must cite exact evidence from at least one supplied successful trajectory: trajectory id, step, an environment-accepted action quote, and a pre- or post-observation quote. Never use an invalid action as positive source evidence.
3. A statement may be more abstract than its quote, but it must be entailed by the cited action and environment observation.
4. Do not assign action scores, advantages, or numeric weights. Do not diagnose trajectories that are not supplied.
5. If the evidence is insufficient, return fewer items or an empty list.

Return one JSON object only:

```
{ "evidence": [{
  "evidence_id": "E1",
  "kind": "requirement|progress|constraint|alternative",
  "statement": "concise natural-language guidance",
  "sources": [{
    "traj_uid": "...", "step": 0,
    "pre_quote": "exact short quote or empty string",
    "action_quote": "exact action quote",
    "post_quote": "exact short quote or empty string"
  }]
}] }
```

Prompt 2: Trajectory-level alignment

Align source-supported guidance to each target’s recorded task, actions, and observations. Explicit terminal reward and success labels are not supplied. Recorded observations are immutable.

Rules:

1. For each target, use only its allowed_evidence items and ids. These candidates have supporting evidence from at least one successful trajectory other than the target.
2. Inspect all supplied steps of the realized target trajectory. Check each item’s applicability against the task and recorded states and actions. Source-supported guidance may still be inapplicable to this target path.
3. Base applicability on specific interaction evidence. Do not judge an action solely from an apparent overall outcome or output inferred terminal labels.
4. Every retained statement must cite allowed source evidence ids and exact target support from the task, action, pre-observation, or post-observation.
5. You may reformulate guidance as a target-specific warning or requirement when supported jointly by the cited source and target evidence. Never fabricate observations or counterfactual rollouts. Produce one guidance set for the target trajectory.
6. Do not output action scores, advantages, importance weights, or numeric updates. If the evidence does not support alignment, return empty guidance and an abstention reason.

Return one JSON object only:

```
{ "targets": [{
  "traj_uid": "...",
  "guidance": [{
    "statement": "concise guidance for the shared PI",
    "evidence_ids": ["E1"],
    "target_support": [{
      "step": 2, "field": "task|pre|action|post",
      "quote": "exact short quote"
    }]
  }],
  "abstain_reason": "empty when guidance is retained"
}] }
```

Validation. Responses must be JSON objects. Source items require unique nonempty IDs, nonempty statements of at most 800 characters, and references to valid actions in verified successes. Each reference must match the action quote and a pre- or post-observation quote. Alignment requires a

known target, a nonempty statement of at most 800 characters, allowed evidence IDs, and a supported target quote. We check quotes by whitespace-normalized substring matching against original records. These checks establish traceability; entailment and applicability remain analyzer judgments. Rejected items are discarded; empty guidance yields no PI.

Context compaction. All recorded steps are retained. After whitespace normalization, overlong fields keep their beginning and end, separated by `. . .`. The character budget C is distributed across the task and steps: task/action caps are 1,200/320 characters and the minimum observation budget is 40 characters. Field minima and JSON overhead make C an approximate request budget.

PI composition and budgeting. Accepted statements among the first G candidates are joined as bullets and shared across turns. We use token budget B_{PI} to distinguish it from trajectory count B . If [Privileged Guidance] and [End Privileged Guidance] occupy $B_{wrapper} < B_{PI}$ tokens, they enclose the first $B_{PI} - B_{wrapper}$ guidance tokens. Otherwise, we use the first B_{PI} guidance tokens alone. Original contexts and sampled responses are preserved. Truncation may cut an item: reference checks apply to the original statement, without guaranteeing completeness of the truncated prefix.

Defaults. The configuration defaults are $C = 12,000$, $G = 8$, $B_{PI} = 256$, and an analyzer output limit of 4,096 tokens. The scoring defaults are $c_{\Delta} = 2.0$, $\tau = 0.5$, and $\epsilon_w = 0.4$.

A.3 SCORING AND LOSS IMPLEMENTATION

Action spans. We locate the first complete `<action>`, `<search>`, or `<answer>` block, matching paired tags case-insensitively across lines. After decoding response tokens individually, we include tokens overlapping the matched span, including both tags. If no block is found, the same pattern searches the recorded response text. The complete matched tagged block is tokenized, with or without one leading space, and located as an exact response-token subsequence. Failure yields an empty action mask.

Paired scoring and valid turns. Both passes use the frozen rollout snapshot and original sampled response, adding P_i only to the privileged context. A turn enters $\mathcal{I}_i$ when both inputs are available, $\mathcal{M}_{it}$ is complete and nonempty, and every action-token log probability and difference Δ_{itk} from Eq. equation 3 is finite. Otherwise, we exclude the turn to avoid comparing partial and complete action scores.

Clipped action means. For $t \in \mathcal{I}_i$, we compute

$$d_{it} = \frac{1}{|\mathcal{M}_{it}|} \sum_{k \in \mathcal{M}_{it}} \text{clip}(\Delta_{itk}, -c_{\Delta}, c_{\Delta}), \quad c_{\Delta} > 0. \quad (\text{A.1})$$

We apply no small-magnitude filter and average over the complete action. Both centering operations use the same $\mathcal{I}_i$: d_{it} before $\tanh$, and q_{it} when constructing weights.

Local and trajectory fallback. Unscorable turns keep $w_{it} = 1$. No shared PI, $A_i = 0$, or $|\mathcal{I}_i| < 2$ leaves all weights at one. Otherwise, redistribution proceeds within $\mathcal{I}_i$ and preserves the full trajectory mean, as shown in Appendix B.1.

Loss normalization. Action-derived weights apply to all generated response tokens, including reasoning; observations and padding are masked out. This does not independently assess reasoning correctness. To implement Eq. equation 7 with row averaging, multiply each real turn’s token-averaged term by $N_{\text{row}}/(N_{\text{traj}}T_i)$. N_{row} includes padded rows, which contribute zero; N_{traj} counts real trajectories. This implements the nested averaging in the main text. The balancing factor remains active during fallback and is separate from the mean-one credit weights.

B DERIVATION AND PROPERTIES

B.1 WEIGHT DERIVATION AND CREDIT PRESERVATION

For a fixed trajectory with $A_i \neq 0$ and $n = |\mathcal{I}_i| \geq 2$, let $q_{it} = \tanh(\text{sign}(A_i)\hat{d}_{it}/\tau)$ and $\eta = \epsilon_w/2$. The weight rule admits the following constrained allocation interpretation:

$$\begin{aligned} \max_{\{w_{it}\}_{t \in \mathcal{I}_i}} \quad & \eta \sum_{t \in \mathcal{I}_i} q_{it}(w_{it} - 1) - \frac{1}{2} \sum_{t \in \mathcal{I}_i} (w_{it} - 1)^2 \\ \text{s.t.} \quad & \sum_{t \in \mathcal{I}_i} (w_{it} - 1) = 0, \quad |w_{it} - 1| \leq \epsilon_w. \end{aligned} \quad (\text{B.1})$$

The linear term favors larger weights for larger q_{it} , while the quadratic term penalizes departures from uniform credit.

Derivation. With only the equality constraint, stationarity gives $w_{it} - 1 = \eta q_{it} - \lambda$. Summing over $\mathcal{I}_i$ gives $\lambda = \eta \bar{q}_i$, where $\bar{q}_i = n^{-1} \sum_{t \in \mathcal{I}_i} q_{it}$. Therefore,

$$w_{it}^* = 1 + \eta(q_{it} - \bar{q}_i). \quad (\text{B.2})$$

Since $q_{it}, \bar{q}_i \in [-1, 1]$, $|w_{it}^* - 1| \leq 2\eta = \epsilon_w$. The solution already satisfies the box constraints. Strict concavity therefore makes it the unique optimizer of Eq. equation B.1, recovering Eq. equation 6 without an additional optimization step.

Preserved quantities. Use Eq. equation B.2 on scorable turns and unit weights elsewhere. The centered corrections sum to zero within $\mathcal{I}_i$, so the complete trajectory has $\sum_{t=1}^{T_i} w_{it} = n + (T_i - n) = T_i$. For $0 \leq \epsilon_w < 1$,

$$1 - \epsilon_w \leq w_{it} \leq 1 + \epsilon_w, \quad w_{it} > 0, \quad \frac{1}{T_i} \sum_{t=1}^{T_i} w_{it} = 1. \quad (\text{B.3})$$

Consequently,

$$\text{sign}(\tilde{A}_{it}) = \text{sign}(A_i), \quad \frac{1}{T_i} \sum_{t=1}^{T_i} \tilde{A}_{it} = A_i, \quad (\text{B.4})$$

$$|\tilde{A}_{it} - A_i| \leq \epsilon_w |A_i|.$$

The unit-weight fallback satisfies the same properties. In particular, an action on a negative-advantage trajectory retains negative credit even when guidance supports it; redistribution can attenuate that credit without reversing its sign. These are scalar advantage guarantees. The parameter-gradient direction also depends on the individual token gradients and can change under reweighting.

B.2 CLIPPED SCORES AND RELATIVE SUPPORT

Bounded token influence. Consider a fixed scorable action with L tokens and let $f_c(z) = \text{clip}(z, -c, c)$, with $c = c_\Delta$. Since $|f_c(z)| \leq c$,

$$d = \frac{1}{L} \sum_{k=1}^L f_c(\Delta_k), \quad |d| \leq c. \quad (\text{B.5})$$

If only token j changes from Δ_j to Δ'_j , clipping is 1-Lipschitz and has range $[-c, c]$, giving

$$|d' - d| \leq \frac{1}{L} \min\{|\Delta'_j - \Delta_j|, 2c\}. \quad (\text{B.6})$$

Thus, one token changes the action mean by at most $2c/L$, assuming a fixed action span and unchanged scoring validity. The bound makes no assumption about which shifts reflect task value.

Shared-shift invariance. For fixed $\mathcal{I}_i$, add a constant b_i to every aggregated action score: $d'_{it} = d_{it} + b_i$. Then $d'_{it} - \bar{d}'_i = (d_{it} + b_i) - (\bar{d}_i + b_i) = \hat{d}_{it}$, so q_{it} and w_{it} remain unchanged. This applies to aggregated scores; shared offsets before token clipping need not cancel because clipping is nonlinear. The two centerings establish relative support and mean-one weights, respectively.

B.3 COMPATIBILITY WITH THE POLICY OBJECTIVE

Let r be the current-to-rollout policy probability ratio for a sampled token under its original context. Write the clipped surrogate to be maximized as $s_{\text{clip}}(r, A) = \min\{rA, \text{clip}(r, 1 - \epsilon_{\text{PPO}}, 1 + \epsilon_{\text{PPO}})A\}$. For $A < 0$, define the dual-clipped surrogate $s_{\text{dc}}(r, A) = \max\{s_{\text{clip}}(r, A), c_{\text{dc}}A\}$ with $c_{\text{dc}} > 1$; otherwise it equals $s_{\text{clip}}(r, A)$. For a fixed detached $w > 0$,

$$s_{\text{clip}}(r, wA) = w s_{\text{clip}}(r, A), \quad s_{\text{dc}}(r, wA) = w s_{\text{dc}}(r, A). \quad (\text{B.7})$$

Positive homogeneity of minimum and maximum gives both identities; the dual-clipped sign condition is also preserved. They apply equally to the negative-surrogate policy loss. At fixed r , the clipping branch is unchanged. KL and entropy retain their separate coefficients.

C EXPERIMENTAL DETAILS

C.1 EVALUATION PROTOCOLS

Table C.1 summarizes the evaluation sets. Results use the prescribed final checkpoint and three training seeds; held-out tasks are excluded from training and checkpoint selection.

Table C.1: Training data and evaluation sets.

Domain	Training data	Evaluation	Metrics
ALFWorld	Training split	140 seen; 134 unseen tasks	Success rate
WebShop	Training split	128 fixed tasks	Task score; success rate
Search-QA	NQ and HotpotQA	Seven datasets listed in the main text	Exact match

C.2 TRAINING AND TASPO CONFIGURATION

We use Qwen2.5-3B-Instruct, Qwen2.5-7B-Instruct, and Qwen3-1.7B-Instruct; controlled analyses use Qwen2.5-3B-Instruct unless specified otherwise. Tables C.2 and C.3 collect the training settings.

Table C.2: Policy optimization and rollout configuration. Shared settings span all columns.

Setting	ALFWorld	WebShop	Search-QA
Tasks per update	16	16	128
Maximum turns	50	15	4
Maximum prompt tokens	2,048	4,096	4,096
KL coefficient	0.01	0.01	0.001
Warmup ratio	0	0	0.1
PPO minibatch size	256	64	256
Microbatch size per GPU	16	8	16
Optimizer	AdamW		
Learning rate	10^{-6}		
Optimizer (β_1, β_2)	(0.9, 0.999)		
Weight decay / gradient clipping	0.01 / 1.0		
PPO clipping / dual-clip coefficient	0.2 / 3.0		
Policy epochs per batch	1		
Entropy coefficient	0.001		
KL estimator	Low-variance KL		
Rollouts per task / policy updates	8 / 150		
Rollout temperature / top- p	1.0 / 1.0		
Maximum response tokens	512		
Number of training seeds	3		

Table C.3: TASPO-specific settings.

Setting	Value
Default analyzer	DeepSeek-V4-Pro
Analyzer temperature / reasoning mode	0 / disabled
Analyzer output limit / workers	4,096 tokens / 4
Maximum evidence items	12
PI token budget	256
Score temperature τ	0.5
Weight range ϵ_w	0.4
Token-shift threshold κ	2.0

D SUPPLEMENTARY EXPERIMENTS

We examine how each design choice helps reconcile process supervision with outcome-based credit. We first isolate the contribution of target alignment, then compare two ways of using the aligned supervision, and finally examine where the resulting credit should be applied. We use ALFWorld with Qwen2.5-3B-Instruct and follow the main experimental protocol of 150 policy updates and three training seeds.

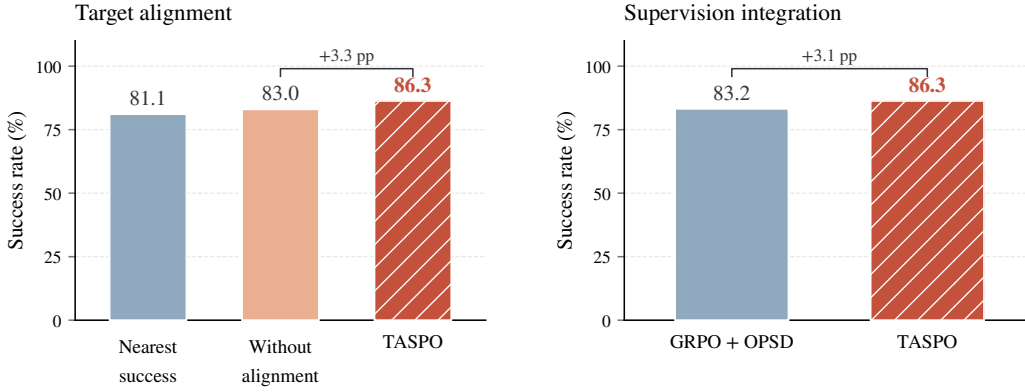


Figure D.1: **Target alignment and supervision integration.** ALFWorld success rates with Qwen2.5-3B-Instruct. Left: contribution of target alignment beyond guidance extraction. Right: auxiliary distillation versus credit redistribution under the same aligned-PI construction protocol.

D.1 CONTRIBUTION OF TARGET-TRAJECTORY ALIGNMENT

We first test whether extracting conditional guidance is sufficient, or whether its applicability must also be checked against the target trajectory. To isolate this distinction, we remove target alignment while retaining guidance extraction, source-evidence checks, exclusion of target-only evidence, and the same PI budget and credit allocation rule.

As shown in Figure D.1(a), abstracting source guidance improves success from 81.1% with nearest-success reuse to 83.0%. Adding target alignment further improves it to 86.3%. We thus find complementary gains from the two stages: abstraction expresses the conditions behind successful source actions, while alignment checks whether those conditions apply to the target’s recorded interaction. The additional 3.3-point gain supports checking applicability before using process supervision to allocate credit.

D.2 INTEGRATING ALIGNED PROCESS SUPERVISION

We next examine whether the benefit comes only from better guidance or also from how that guidance enters optimization. We equip GRPO+OPSD with the same analyzer, extraction and alignment

prompts, PI budget, and scoring procedure with a frozen rollout policy as TASPO. Each method constructs PI from its own on-policy trajectories. GRPO+OPSD uses the resulting supervision in an auxiliary distillation loss, whereas TASPO uses it to redistribute the original outcome advantage.

Figure D.1(b) shows that TASPO reaches 86.3% success, compared with 83.2% for GRPO+OPSD with aligned PI. We therefore observe a benefit from credit redistribution even after matching the supervision-construction protocol. This comparison supports our central design choice: process information helps determine where to allocate outcome-based credit, while the verified outcome continues to determine the sign of the advantage.

D.3 SCOPE OF ACTION-DERIVED CREDIT

We score executable actions but apply the resulting coefficient to the full response that produces each action. To examine this choice, we compare it with an action-only variant that retains the original advantage A_i on reasoning tokens. Both variants use action-level scores; we change only the scope of credit application and the centering needed to preserve mean credit.

Under our turn-averaged loss, directly masking the original weights to action tokens would generally change the trajectory-average advantage. We therefore account for the action-token fraction $\rho_{it} = |\mathcal{M}_{it}|/|\mathcal{R}_{it}|$, where $\mathcal{R}_{it}$ contains all generated response-token positions. Using the same q_{it} as TASPO, we define the action-only weights as

$$\begin{aligned}\bar{q}_i^{\text{act}} &= \frac{\sum_{t \in \mathcal{I}_i} \rho_{it} q_{it}}{\sum_{t \in \mathcal{I}_i} \rho_{it}}, \\ w_{it}^{\text{act}} &= 1 + \frac{\epsilon_w}{2} (q_{it} - \bar{q}_i^{\text{act}}).\end{aligned}\tag{D.1}$$

We apply these weights only to action tokens, retain unit weights elsewhere, and use the same validity checks and fallback rules. This gives $\sum_{t \in \mathcal{I}_i} \rho_{it} (w_{it}^{\text{act}} - 1) = 0$, preserving mean credit under the same loss reduction and keeping the weights positive and bounded.

Table D.1: **Scope of action-derived credit.** Both variants score executable actions and preserve mean credit.

Configuration	Rewighted tokens	Success (%)
Action-only	Action tokens	84.0
TASPO	Full response	86.3

Table D.1 shows a 2.3-point advantage for full-response allocation. Under the same per-token weight bounds, we obtain better task performance by adjusting the response that produces an action as a whole. We use this result to justify the application scope; action-derived scores do not separately assess the correctness of the reasoning trace.

D.4 COMPUTATIONAL COST

We use additional analysis computation to obtain more supervision from an existing rollout batch. This adds guidance extraction and alignment by the analyzer, followed by PI-conditioned policy scoring. We reuse the original-view log probabilities when they come from the same frozen rollout snapshot and use the same scoring definition. Neither PI construction nor rescoring requires additional environment rollouts.

With N_{upd} updates, B_{task} tasks per update, and K rollouts per task, the scheduled trajectory count remains $N_{\text{upd}} B_{\text{task}} K$: 19,200 for ALFWorld and WebShop, and 153,600 for Search-QA. We profile an ALFWorld run with Qwen2.5-3B-Instruct on eight H800 GPUs through update 150. Including one replayed batch, the run contains 151 completed batches, 19,328 trajectories, 508,704 environment steps, and 2,974 analyzer request attempts.

Figure D.2 shows selected stage runtimes. PI construction takes 4.50 hours, while the additional scoring pass requires 0.23 hours, or about 5.4 seconds per completed batch. Most of the added computation therefore lies in constructing the guidance, with a small additional scoring cost. In the main ALFWorld evaluation with the same backbone, TASPO improves success from 71.8% to 86.3%,

a gain of 14.5 percentage points under the same scheduled rollout and policy-update budgets. We view the added analysis cost as a worthwhile trade-off when interaction budgets are constrained: it improves policy learning by extracting more supervision from trajectories already collected.

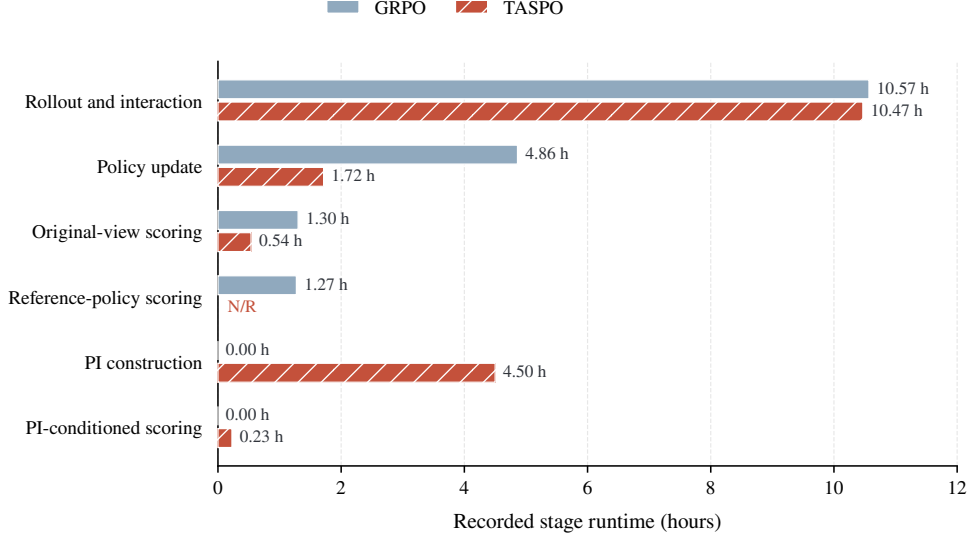


Figure D.2: **Selected training-stage runtimes on ALFWorld.** Both runs use Qwen2.5-3B-Instruct through update 150. GRPO uses four GPUs of an unconfirmed model; TASPO uses eight H800 GPUs. The bars report individual stages rather than total runtime. N/R denotes a stage not separately reported. Initialization, evaluation, and checkpoint saving are excluded.

E QUALITATIVE ANALYSIS

Recovery from an Ineffective Action. We compare GRPO and TASPO on the same ALFWorld task instance under matched evaluation settings. The task requires the agent to heat a cup and place it in a cabinet. Figure E.1 shows how the two policies respond when an attempted interaction fails.

After heating the cup, both agents attempt to open `cabinet 1` and receive “Nothing happens.” Their subsequent actions reveal the difference. GRPO repeats the opening command without returning to the cabinet, making no progress toward the placement goal and ultimately failing the task. TASPO instead executes `go to cabinet 1` before retrying `open cabinet 1`, which succeeds. It subsequently places the heated cup in `cabinet 16` and completes the task. In this case, the distinction lies in how the policies proceed after an error: TASPO changes the interaction state before retrying, allowing execution to resume.

The successful trajectory also contains an ineffective action. This makes the case informative for credit assignment: the failed opening attempt, the navigation that enables a successful retry, and the final placement all share the same positive trajectory advantage under outcome-only allocation. Our design introduces aligned process supervision to distinguish the relative support for these decisions while retaining the outcome advantage’s sign and trajectory mean. The observed recovery illustrates the behavior this finer allocation is intended to support.

Revising Product Selection under Task Constraints. We next compare GRPO and TASPO on the same WebShop task. The task requests a blue 32 oz water bottle priced below \$30.00. As shown in Figure E.2, both agents initially inspect a product that satisfies the color and price requirements but offers only a 16 oz capacity. Its detail page therefore provides sufficient information to rule out this candidate.

The policies diverge after inspecting these details. GRPO selects the requested color and completes the purchase, leaving the capacity requirement unmet. TASPO returns to search and reformulates its query around the required capacity. It then inspects another product, selects the requested color,

Task and shared decision point	
Task: Heat some cup and put it in cabinet.	
The recorded prefix has reached <code>microwave 1</code> and heated <code>cup 3</code> .	
Environment (Step 12)	
You heat the <code>cup 3</code> using the <code>microwave 1</code> .	
Agent (Step 12)	
<code>open cabinet 1</code>	
Environment (Step 13)	
Nothing happens.	
Failure continuation	Recorded recovery continuation
Agent (Step 13)	Agent (Step 13)
<code>open cabinet 1</code>	<code>go to cabinet 1</code>
Environment (Step 14)	Environment (Step 14)
Nothing happens.	You arrive at cabinet 1. The cabinet 1 is closed.
Agent (Step 14)	Agent (Step 14)
<code>open cabinet 1</code>	<code>open cabinet 1</code>
Environment (Step 15)	Environment (Step 15)
Nothing happens.	You open the cabinet 1. The cabinet 1 is open. In it, you see nothing.
Unresolved execution condition	
The agent repeats the operation without first navigating to the cabinet. Under the assumed unchanged state, these attempts make no progress.	
Continued repetition would consume the interaction budget while leaving the placement goal unmet.	
Steps 15–18: recorded actions, condensed	
The agent visits <code>countertop 1</code> , checks its inventory, navigates to <code>cabinet 16</code> , and opens it.	
Environment (Step 19)	
You open the cabinet 16. The cabinet 16 is open. In it, you see nothing.	
Agent (Step 19)	
<code>move cup 3 to cabinet 16</code>	
Recorded outcome	
Task success: True .	

Figure E.1: **Recovery after an unsuccessful action on ALFWorld.** After an unsuccessful attempt to open a cabinet, GRPO (left) repeats the operation without first navigating to the cabinet. **TASPO (right)** instead navigates to the cabinet before retrying, successfully opens it, and ultimately completes the task. The right branch shows selected interactions from a Qwen2.5-3B-Instruct rollout; Steps 15–18 are condensed.

and completes a purchase that satisfies all stated constraints. The consequential decision is to leave the first candidate: although returning to search does not immediately complete the task, it enables a suitable product to be found.

Here, the shopping interface permits the purchase, so successful execution alone does not establish that an action advances the task. The decision must also account for the requested attributes and the options available on the current page. This is the distinction our alignment stage is designed to capture: guidance about selecting a particular option applies only when that option is available in the target interaction. We check such conditions against the recorded target history before using process supervision to redistribute outcome credit. The comparison illustrates the value of revising an intermediate choice when new information reveals that it cannot satisfy the final goal.

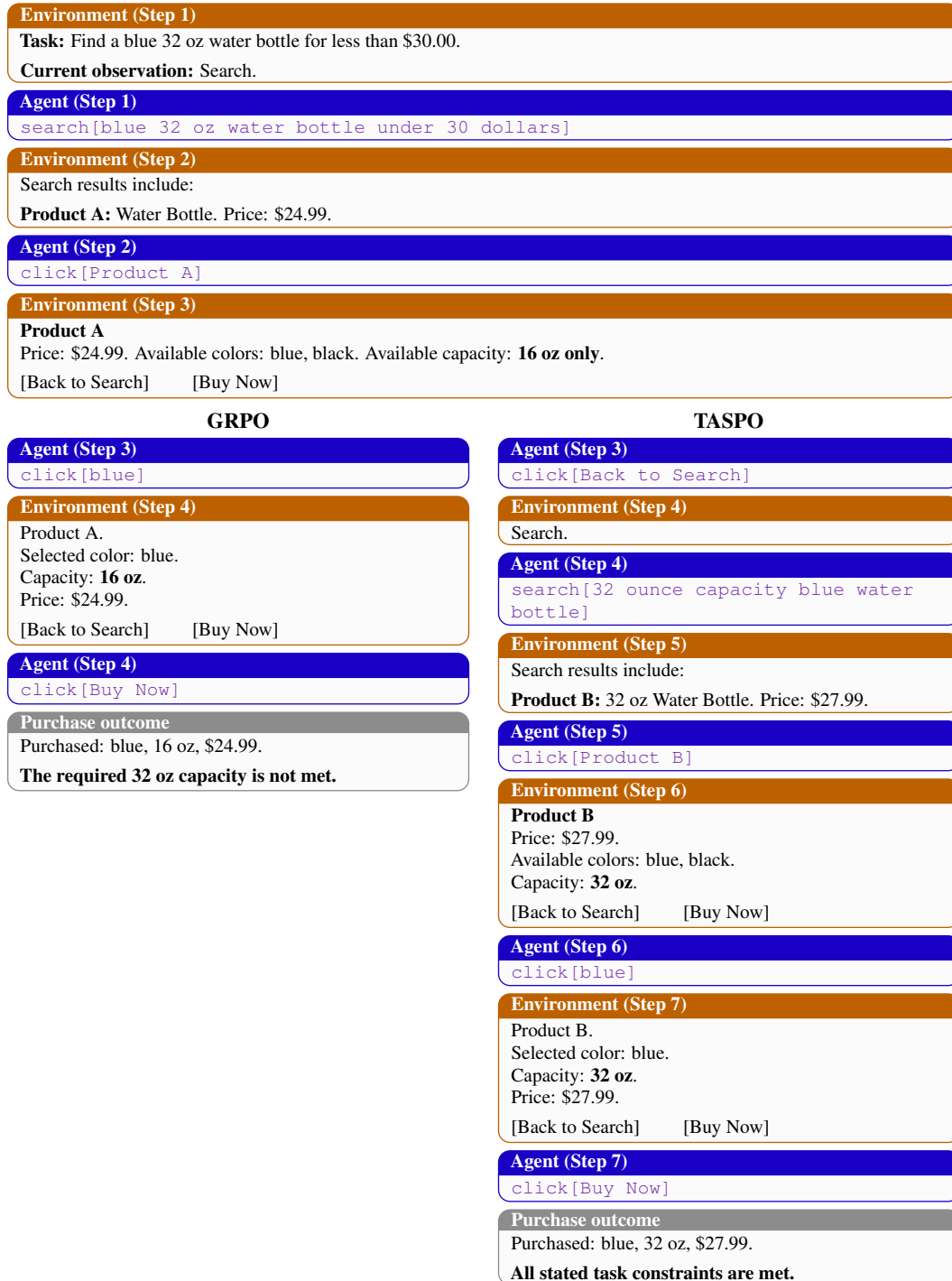


Figure E.2: **Revising product selection on WebShop.** The first candidate matches the requested color and price but lacks the required capacity. The two branches contrast purchasing this candidate with returning to search and selecting a suitable alternative.